

DYAD FINANCE

Arquitectura CQRS (Command Query Responsibility Segregation)

Rol: Software Architect & Web3 Developer

Autor: Antigravity AI Partner

Fecha: Junio 2026

Version: 1.0 (Produccion)

1. Que es CQRS y por que aplicarlo a Dyad Finance?

CQRS (Segregacion de Responsabilidades de Consulta y Comando) es un patron de arquitectura que separa estrictamente las operaciones que modifican el estado (Comandos / Commands) de las operaciones que solo leen informacion (Consultas / Queries).

En el desarrollo de aplicaciones descentralizadas (dApps) como Dyad Finance, esta separacion es critica debido a la naturaleza asincrona y costosa de la blockchain:

- Comandos (Escrituras): Son transacciones que alteran la blockchain (ej. depositar ETH, mintear usdJ, bloquear un RWA NFT). Son lentas (dependen de la confirmacion del bloque), requieren pago de gas, pueden ser rechazadas por el usuario en MetaMask, y su exito debe ser monitoreado en segundo plano.
- Consultas (Lecturas): Obtienen balances, tasas de mineria, y precios del oraculo. Deben ser instantaneas, no cuestan gas y deben cachearse localmente para evitar exceder las cuotas de peticiones RPC del nodo Web3 (ej. Alchemy/Infura).

Al aplicar CQRS en el frontend de Dyad, independizamos ambos flujos, previniendo que llamadas lentas de red congelen la UI o que errores de transacciones interrumpen la visualizacion de saldos.

2. El Flujo de Comandos y Consultas en Dyad

En una arquitectura CQRS tradicional en dApps, dividimos las intenciones del usuario de la siguiente manera:

A. El Camino de los Comandos (Writes):

1. El usuario presiona 'Stake NFT' o 'Depositar ETH'.
2. El UI despacha un objeto de Comando (ej. 'StakeNftCommand').
3. El Command Handler valida el saldo y envia la peticion a MetaMask.
4. MetaMask solicita la firma. Si se confirma, se emite una transaccion Web3.
5. El Handler registra la transaccion en un monitor asincrono y retorna un ID temporal.
6. Tras la confirmacion on-chain, se emite un Evento (ej. 'NftStakedEvent') que invalida las queries.

B. El Camino de las Consultas (Reads):

1. El UI solicita los datos de balance o tasa de mineria mediante una Query (ej. 'GetBalancesQuery').
2. El Query Handler revisa primero la cache en memoria (React Query / LocalState).
3. Si los datos estan expirados, realiza una llamada rapida de lectura RPC a la blockchain (sin coste de gas).
4. Retorna los datos y los guarda en cache, evitando llamadas repetidas en re-renders.

3. Mapeo Tactico de Componentes en Dyad

Catalogo de Comandos (Write Model)

- DepositCollateralCommand(amount): Deposita ETH para acuñar usdJ y jETH.
- RedeemStablecoinCommand(amount): Canjea usdJ para recuperar ETH.
- RegisterRwaPropertyCommand(propertyId, docs): Envía pruebas de avance de obra al oráculo.
- StakeRwaNftCommand(nftId): Bloquea el NFT de RWA en el pool de rendimiento.
- ClaimMiningRewardsCommand(): Extrae las ganancias acumuladas jKERO del pool.
- UnstakeRwaNftCommand(nftId): Retira el NFT de staking y liquida recompensas.

Catalogo de Consultas (Read Model)

- GetTokenBalancesQuery(account): Consulta rápida de balances (ETH, usdJ, jETH, USDC).
- GetAccountHealthQuery(account): Calcula el ratio de colateralización y límite de LTV.
- GetRealEstateStagesQuery(): Consulta los estados físicos de las propiedades (Gris, 3D, Completo).
- GetAccumulatedMiningYieldQuery(account): Lee las recompensas acumuladas en tiempo real.
- GetActiveMultiplierQuery(stage): Retorna el multiplicador actual de incentivo.

4. Propuesta de Organización en el Proyecto

```
src/application/
-- shared/
  -- bus/                                # Bus de comandos y consultas en memoria
    -- command-bus.ts
    -- query-bus.ts
-- vault/
  -- commands/                          # Modificaciones de estado
    -- deposit-collateral.command.ts
    -- deposit-collateral.handler.ts
  -- queries/                           # Consultas de lectura
    -- get-balances.query.ts
    -- get-balances.handler.ts
-- yield/
  -- commands/                          # Staking, Claim y Unstake
    -- stake-rwa.command.ts
    -- claim-rewards.command.ts
  -- queries/
    -- get-accumulated-yield.query.ts
```

5. Ejemplo de Implementacion de Comando en TypeScript

```
// deposit-collateral.command.ts
export class DepositCollateralCommand {
  constructor(public readonly amountInEth: string) {}
}

// deposit-collateral.handler.ts
export class DepositCollateralHandler {
  constructor(
    private readonly walletService: IWalletService,
    private readonly vaultContract: IVaultContract
  ) {}

  async handle(command: DepositCollateralCommand): Promise<string> {
    const signer = await this.walletService.getSigner();
    const tx = await this.vaultContract.deposit(signer, command.amountInEth);
    // Retornamos el hash de transaccion temporal de inmediato
    return tx.hash;
  }
}
```

6. Conclusiones y Beneficios para el Equipo

Al separar las lecturas de las escrituras cripto, logramos:

1. Experiencia de Usuario Ininterrumpida: La interfaz puede mostrar estados temporales mientras los comandos asincronos Web3 se confirman on-chain.
2. Escalabilidad en Lectura: Permite en un futuro usar indexadores mas eficientes como The Graph o cachés de Redis sin tocar una sola linea de la logica de las transacciones.
3. Testabilidad Limpia: Los Query Handlers se testean verificando los retornos de datos, y los Command Handlers verificando el envio correcto de parametros Web3.